

CSEE 3827: Fundamentals of Computer Systems, Spring 2026

Lecture 2

Prof. Dan Rubenstein (danr@cs.columbia.edu)





Boolean Logic is
Coming.....

Agenda (M&K 2.1-2.2)

- Terminology
- Boolean algebra (incl. DeMorgan's Law)
- Logic gates
- Circuit fabrication
 - NAND, NOR
- DUAL
- XOR

Boolean Terminology

Terminology

- **Recall: Digital / Binary / Boolean:** 0 = False, 1 = True
- Binary or Boolean **Variable**: a symbolic representation of a value that might be 0 or 1, e.g., X, Y, A, B
- **Complement** (e.g., of a variable X): written $\bar{X}$: the opposite value of X

X	$\bar{X}$
0	1
1	0

I sometimes say “negate”: , e.g.,
if boolean $X=1$, negating X makes $X=0$.
Negating X again changes it back to 1

- **Literal**: a boolean variable or its complement (e.g., X, $\bar{X}$, $\bar{Y}$)

Boolean Logic

- All logical functions can be implemented in terms of three logical operations:

NOT

x	$\bar{x}$
0	1
1	0

AND

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

can omit the “.”

OR

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

Not addition anymore!

- Whether “+” represents addition or logical OR can often be gotten from context
- When not from context, we will specify

Boolean Logic

- All logical functions can be implemented in terms of three logical operations:

NOT

x	$\bar{x}$
0	1
1	0

AND

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

can omit the “.”

OR

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

Not addition anymore!

The the “OR” is true when both are true!

- Whether “+” represents addition or logical OR can often be gotten from context
- When not from context, we will specify

Boolean Logic 2

- Boolean logic precedence rules just like addition / multiplication
- Implied precedence: NOT before AND before OR
- Use parentheses as necessary to emphasize precedence, or force alternate precedence

$A+BC$ same as $A+(BC)$

$(\overline{A} + B)C$ same as $((\overline{A}) + B)C$

$(A + B) C$ differs from $A + BC$

Terminology cont'd

- **Expression**: a set of literals (possibly with repeats) combined with logic operations (and possibly ordered by parentheses)

- e.g., 4 expressions: $AB + C, (AB) + C, (\overline{A} + B)C, ((\overline{A}) + B)C$

- **Equation**: expression1 = expression2

- e.g., $(\overline{A} + B)C = ((\overline{A}) + B)C$

- **Function** of (possibly several) variables: an equation where the lefthand side is defined by the righthand side

$$F(A,B,C) = ((\overline{A}) + B)C$$

XOR: the parity operation

- $X \oplus Y = X\bar{Y} + \bar{X}Y$

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0

$X \oplus Y$ true when
X is true or Y is true,
but not both true
(it's exclusive!)

Order of precedence is
similar to OR (+)
(Kind of like + and - in
regular arithmetic)

- In general, represents parity, i.e.,

- $X_1 \oplus X_2 \oplus X_3 \oplus \dots \oplus X_k = 1$ when an odd number of $X_i = 1$

Note: $(X \oplus Y) + Z$ is different
from $X \oplus (Y + Z)$
just as $X - Y + Z$ differs
from $X - (Y + Z)$

XOR as a (parity) “control”

- $Y \oplus 0 = Y$
- $Y \oplus 1 = \overline{Y}$
- As we XOR bits, the result “flips” when XOR’ing a 1, and stays the same when XOR’ing a 0

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0

• $1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 1$
 $= 1$

•

Boolean Logic: Truth Table

D	X	A	$L = \overline{D}X + A$
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Truth Table: Function output for all combinations of input variables

k variables $\rightarrow 2^k$ input combinations

e.g., $L = \overline{D}X + A$

Boolean Logic: Example

Intermediate computations to compute function:

D	X	A	$\overline{X}$	$\overline{DX}$	$L = \overline{DX} + A$
0	0	0	1	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	0	0	1

Boolean Logic: Example 2

Truth table for $XY + \overline{X}\overline{Y}$

X	Y	$XY + \overline{X}\overline{Y}$
0	0	
0	1	
1	0	
1	1	

Boolean Logic: Example 2

Truth table for $XY + \overline{X}\overline{Y}$: intermediate computations

X	Y	$\overline{X}$	$\overline{Y}$	XY	$\overline{X}\overline{Y}$	$XY + \overline{X}\overline{Y}$
0	0	1	1	0	1	1
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	1	0	0	1	0	1

Boolean Theorems

Boolean Algebra: Identities and Theorems

For any boolean X ($X \in \{0,1\}$):

OR	AND	NOT	
$X+0 = X$	$X1 = X$		(identity)
$X+1 = 1$	$X0 = 0$		(null)
$X+X = X$	$XX = X$		(idempotent)
$X+\overline{X} = 1$	$X\overline{X} = 0$		(complementarity)
		$\overline{\overline{X}} = X$	(involution)
$X+Y = Y+X$	$XY = YX$		(commutativity)
$X+(Y+Z) = (X+Y)+Z$	$X(YZ) = (XY)Z$		(associativity)
$X(Y+Z) = XY + XZ$	$(X+Y)(X+Z) = X + YZ$	 Why? We'll see in 2 slides	(distributive)
$\overline{X+Y} = \overline{X} \overline{Y}$	$\overline{XY} = \overline{X} + \overline{Y}$		(DeMorgan's theorem)

Other important simplifications:

- $X + XY = X$:

- If X is false, both sides clearly false
- If X is true, LHS is true no matter what Y is

- $X(X+Y) = X$:

e.g.,

X = “It will snow in March”

Y = “Rubenstein’s hilaaarious”

- If 1st term is false, doesn’t matter what second term is
- If 1st term true, so is 2nd term

Why $(X+Y)(X+Z) = X + YZ$

- We saw $X + XY = X$:
- So $(X+Y)(X+Z) = XX + XZ + YX + YZ$ (*)
- Well, $XX = X$, so $(*) = X + XZ + YX + YZ$
- Well, $X + XZ = X$ (X is true OR (X AND Z) are True is True when & only when X is true)
- So $(*) = X + YX + YZ$
- And for same reason, $X + YX = X$
- so $(*) = X + YZ$

Boolean Algebra: Example

Simplify (reduce # of literals) this equation using algebraic manipulation.

$$F = \overline{X}YZ + \overline{X}Y\overline{Z} + XZ$$

Boolean Algebra: Example

Simplify this equation using algebraic manipulation.

$$F = \overline{X}YZ + \overline{X}Y\overline{Z} + XZ$$

$$\overline{X}Y(Z + \overline{Z}) + XZ \quad \text{(by reverse distribution)}$$

$$\overline{X}Y(1) + XZ \quad \text{(by complementarity)}$$

$$\overline{X}Y + XZ \quad \text{(by identity)}$$

Complementing (parts of) expressions (“the big bar”)

- Complement operations can be applied to expressions (big bars)
- Precedence: the complement is performed after all other boolean ops that lie beneath it
- e.g., $\overline{A + B}$: First, OR A with B, then complement (negate)

A	B	A+B	$\overline{A+B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Complementing (parts of) expressions (“the big bar”)

- Order of precedence example:

$$\overline{\overline{(A + BD)}C}$$

1. Complement A. (In parallel) AND B with D
2. OR $\overline{A}$ with BD
3. Complement $\overline{(\overline{A} + BD)}$
4. AND $\overline{\overline{(\overline{A} + BD)}}$ with C
5. Complement $\overline{\overline{\overline{(\overline{A} + BD)}C}}$

DeMorgan's Theorem

Important!!!!

DeMorgan's Theorem

- Procedure for simplifying complemented expressions (i.e., removal of “big bar”)
- Remove the “big bar” over AND or OR of 2 (or more) functions or variables (e.g., F & G) and replace...
 - AND with OR, OR with AND
 - 1 with 0, 0 with 1
 - function F with $\overline{F}$, $\overline{F}$ with F

$$\overline{FG} = \overline{F} + \overline{G}$$

$$\overline{\overline{F} + \overline{G}} = \overline{\overline{F}} \overline{\overline{G}}$$

Understanding DeMorgan

- Example with boolean variables F & G:
 - F = It will snow (on Saturday)
 - G = Prof. Rubenstein is hilarious

$\overline{FG}$: It's not the case that [It will snow in March
AND Rubenstein's hilaaarious]

$$\overline{FG} = \overline{F} + \overline{G}$$

$\overline{F} + \overline{G}$: It won't snow in March OR Rubenstein's not
hilaaaarious (or both)

$\overline{\overline{F} + \overline{G}}$: It's not the case that [It will snow in March OR
Rubenstein's hilarious (or both)]

$$\overline{\overline{F} + \overline{G}} = \overline{F} \overline{G}$$

$\overline{F} \overline{G}$: It won't snow in March AND Rubenstein's not
hilaaaarious.

DeMorgan's Theorem extended

- Can do for 3 functions (or variables), i.e.,

$$\overline{FGH} = \overline{F} + \overline{G} + \overline{H}$$

$$\overline{F + G + H} = \overline{F}\overline{G}\overline{H}$$

- Can apply to complementing the AND of any N functions (or variables), or complementing the OR of any N functions (or variables)

Boolean Algebra: Example 2

Find the complement of Z.

$$Z = A\bar{B} + \bar{A}B$$

$$\bar{Z} =$$

Boolean Algebra: Example 2

Find the complement of Z.

$$Z = \overset{F}{\underbrace{A\bar{B}}} + \overset{G}{\underbrace{\bar{A}B}}$$

$$\bar{Z} = \overline{A\bar{B} + \bar{A}B}$$

$$\overline{(A\bar{B}) (\bar{A}B)}$$

(by DeMorgan's)

$$(\bar{A} + \bar{\bar{B}}) (\bar{\bar{A}} + \bar{B})$$

(by DeMorgan's)

$$(\bar{A} + B) (A + \bar{B})$$

(by involution)

$$\bar{A}A + \bar{A}\bar{B} + BA + B\bar{B}$$

(by commutativity)

$$\bar{A}\bar{B} + BA$$

(by complementarity and identity)

Boolean Algebra: Example 2

Find the complement of Z.

$$Z = \overset{F}{\underbrace{A\bar{B}}} + \overset{G}{\underbrace{\bar{A}B}}$$

$$\bar{Z} = \overline{A\bar{B} + \bar{A}B}$$

$$\overline{(A\bar{B}) (\bar{A}B)}$$

(by DeMorgan's)

$$(\bar{A} + \bar{\bar{B}}) (\bar{\bar{A}} + \bar{B})$$

(by DeMorgan's)

$$(\bar{A} + B) (A + \bar{B})$$

(by involution)

$$\bar{A}A + \bar{A}\bar{B} + BA + \bar{B}\bar{B}$$

(by commutativity)

$$\bar{A}\bar{B} + BA$$

(by complementarity and identity)

DeMorgan's Practice

$$\overline{\overline{A}\overline{B}\overline{C}} + \overline{\overline{A}\overline{C}\overline{D}} + \overline{B\overline{C}} \longrightarrow F_1 = \overline{\overline{A}\overline{B}\overline{C}}, G_1 = \overline{\overline{A}\overline{C}\overline{D}}, H_1 = \overline{B\overline{C}}, \overline{F_1 + G_1 + H_1} = \overline{F_1} \overline{G_1} \overline{H_1}$$

$$= (\overline{A}\overline{B}\overline{C})(\overline{A}\overline{C}\overline{D})(\overline{B\overline{C}}) \longrightarrow \begin{array}{l} \text{For } \overline{B\overline{C}}: F_2 = B, G_2 = \overline{C}, \overline{F_2 G_2} = \overline{F_2 + G_2} \\ \text{Also, } (\overline{A}\overline{B}\overline{C})(\overline{A}\overline{C}\overline{D}) = \overline{A}\overline{B}\overline{C}\overline{D} \end{array}$$

$$= (\overline{A}\overline{B}\overline{C}\overline{D})(\overline{B} + C)$$

$$= \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D$$

$$= \overline{A}\overline{B}\overline{C}D$$

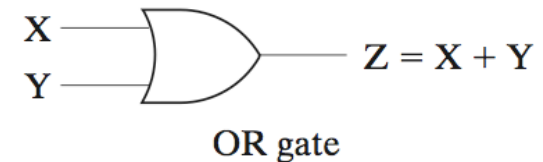
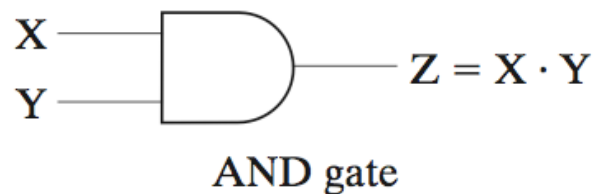
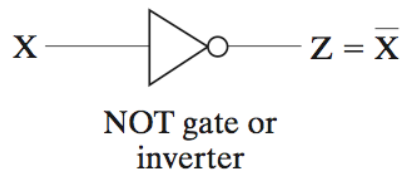
Consensus Theorem

- $XY + \bar{X}Z + YZ = XY + \bar{X}Z$ (The YZ term is redundant)
- Proof:
 - If $YZ=0$, it's obviously not needed (just have $XY+\bar{X}Z+0$ on LHS)
 - If $YZ = 1$
 - both Y and Z are 1. $XY + \bar{X}Z + 1 = 1$ so , LHS equation = 1
 - Either X or $\bar{X} = 1$. Since $Y=Z=1$, either $XY=1$ or $\bar{X}Z = 1$,
 - so RHS equation either $1 + 0$ or $0 + 1 = 1$

Circuit Perspective

Circuit Representation of Boolean Ops

- Information flows from left to right
- Input(s) all the way on the left, output(s) on the right

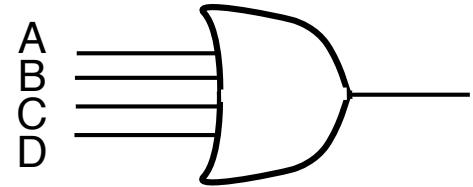


These circuits consume area, power, and time

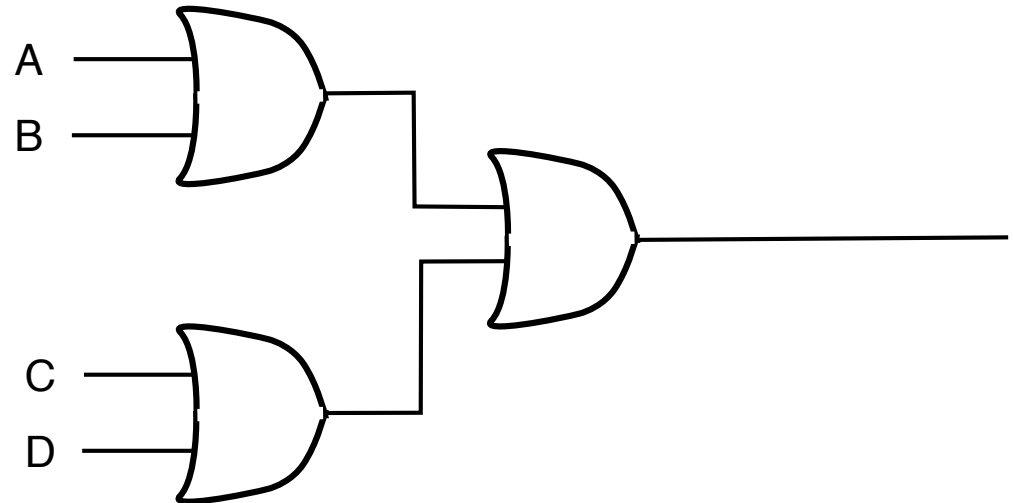
Goal: minimize the amount of circuitry to compute the desired function

more-than-2 input shorthand

- $A+B+C+D$ can be written in shorthand as the gate:



- This is really a series of 2-input gates:



- i.e., a k -input gate is really a sequence of gates with depth $\log_2 k$

We simplify to reduce required circuitry...

$$F = \overline{X}YZ + \overline{X}Y\overline{Z} + XZ$$

$$\overline{X}Y(Z + \overline{Z}) + XZ$$

(by reverse distribution)

$$\overline{X}Y1 + XZ$$

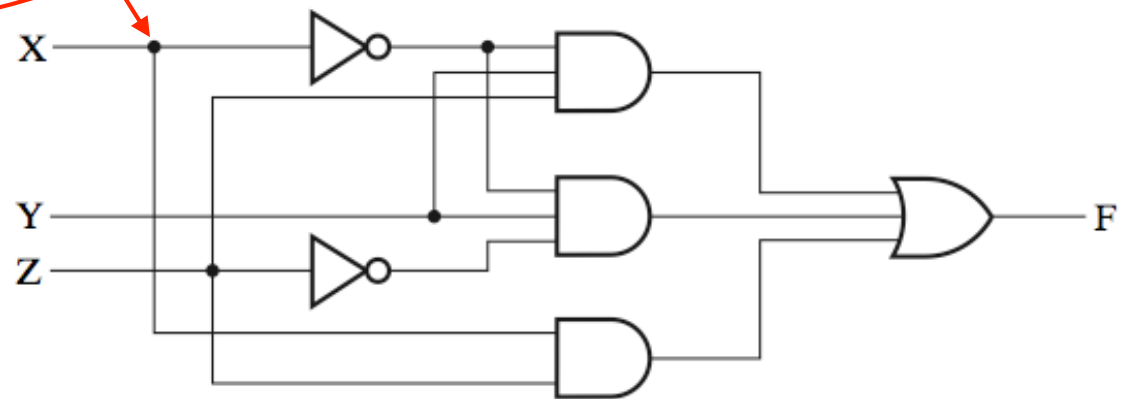
(by complementarity)

$$\overline{X}Y + XZ$$

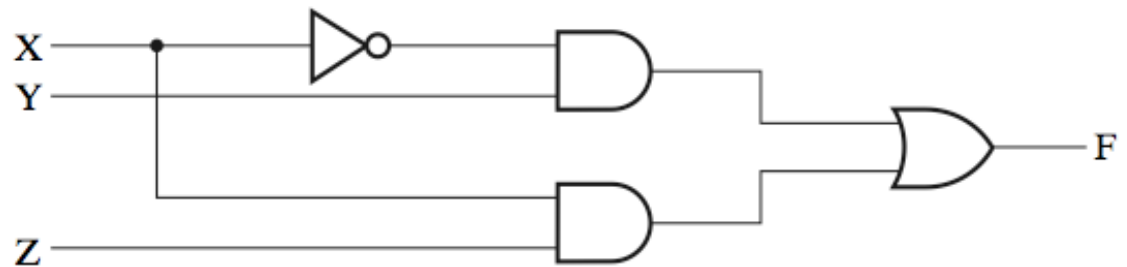
(by identity)

Circuit view

wire connector: black dot signifies wires are connected



(a) $F = \bar{X}YZ + \bar{X}Y\bar{Z} + XZ$



(b) $F = \bar{X}Y + XZ$

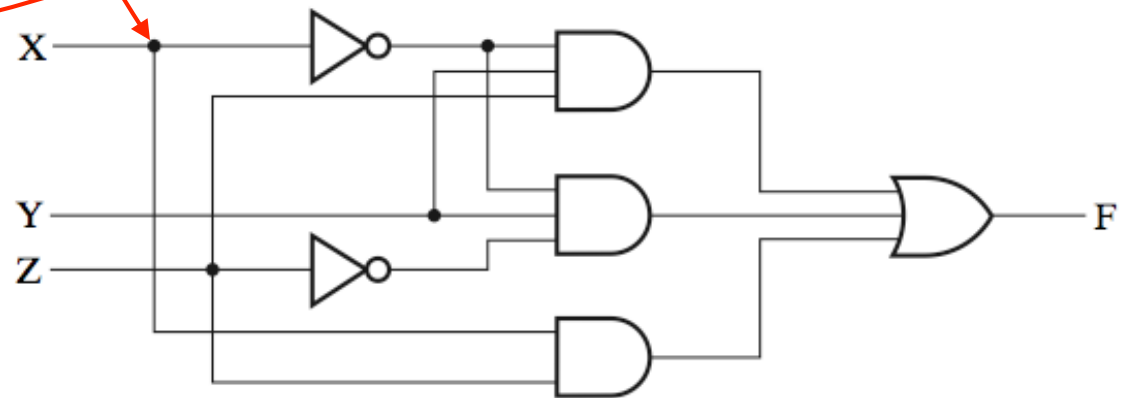
Circuit view: why reduce?

- Cost:

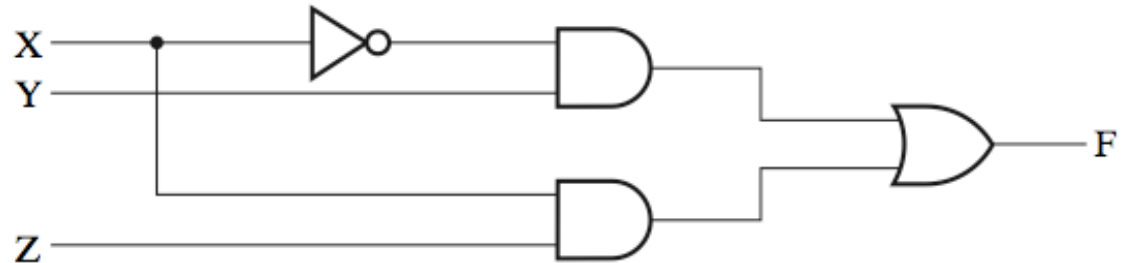
wire connector: black dot signifies wires are connected

- pay per gate:
higher cost
- more gates = more
space on chip

- Wires are fast, gates
are slow: speed of a
circuit (time from
setting inputs to
getting output) roughly
proportional to circuit
depth: longest-gated
path



(a) $F = \bar{X}YZ + \bar{X}Y\bar{Z} + XZ$



(b) $F = \bar{X}Y + XZ$

Top circuit more costly (more gates), and faster when considering 3-input gates each have depth 2

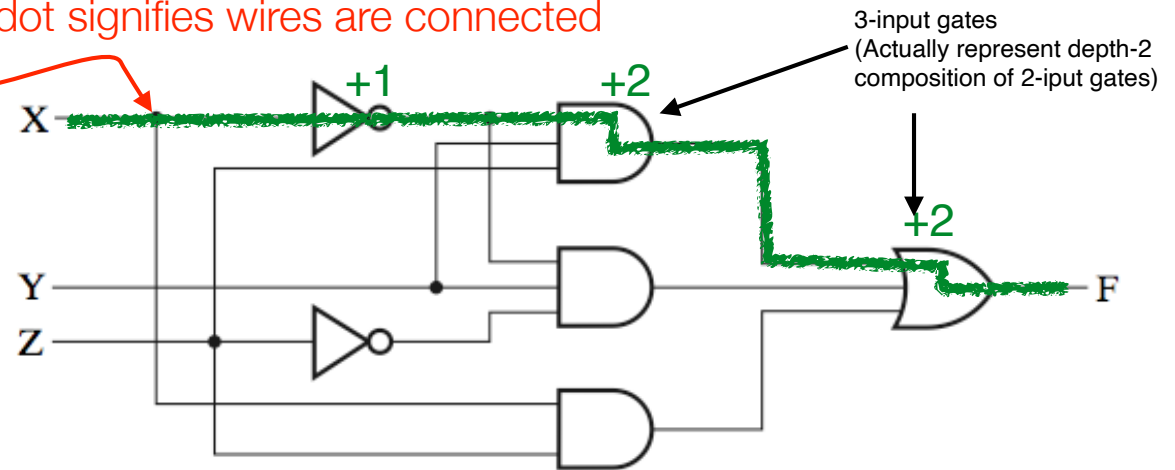
Circuit view: why reduce?

- Cost:

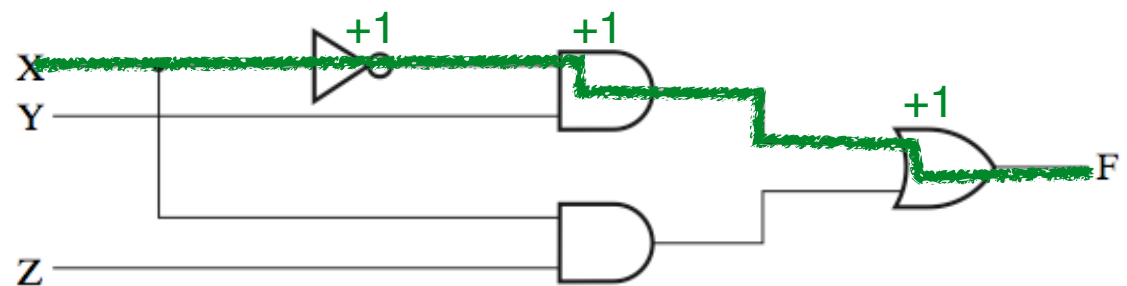
- pay per gate:
higher cost
- more gates = more
space on chip

- Wires are fast, gates
are slow: speed of a
circuit (time from
setting inputs to
getting output) roughly
proportional to circuit
depth: longest-gated
path

wire connector: black dot signifies wires are connected



(a) $F = \bar{X}YZ + \bar{X}Y\bar{Z} + XZ$



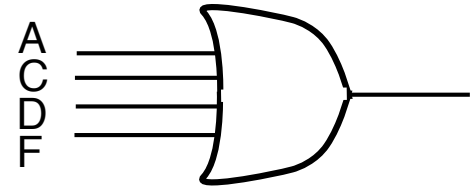
(b) $F = \bar{X}Y + XZ$

Top circuit more costly (more gates), and slower when considering 3-input gates each have depth 2

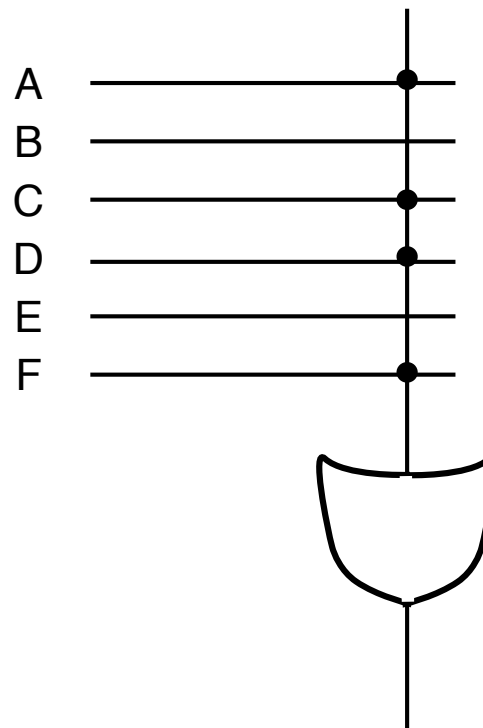
Top circuit depth 5 along top path, bottom circuit depth 3 along top path (longest paths)

Alternate shorthand for multiple inputs to Gate

- Given inputs A,B,C,D,E,F,



- Another representation for $A+C+D+F$ This is really a series of 2-input gates:



black dot signifies connection:
no dot means no connection

Bitwise parallel operations

Bit-wise (in parallel) Logic Ops

- Up to now, we looked at **single-bit** logic ops, e.g., $X + Y = Z$ where X, Y, Z are each one bit
- Will often need to perform logic op on **set of bits** in parallel, e.g.,
 - $X = X_{k-1}X_{k-2}\dots X_1X_0$, $Y = Y_{k-1}Y_{k-2}\dots Y_1Y_0$
 - Want to compute $Z = Z_{k-1}Z_{k-2}\dots Z_1Z_0$ where for each i , $Z_i = X_i + Y_i$
 - Can just write $Z = X + Y$ as a shorthand (here $+$ is logical OR)
 - e.g., $X = 0110$, $Y = 0011$, $Z = X + Y$

$$\begin{array}{r} 0110 \\ + 0011 \\ \hline \end{array}$$

(bitwise logical OR)
 - e.g., $X = 0110$, $Y = 0011$, $Z = XY$

$$\begin{array}{r} 0110 \\ \cdot 0011 \\ \hline \end{array}$$

(bitwise logical AND)

Bit-wise (in parallel) Logic Ops

- Up to now, we looked at **single-bit** logic ops, e.g., $X + Y = Z$ where X, Y, Z are each one bit
- Will often need to perform logic op on **set of bits** in parallel, e.g.,

- $X = X_{k-1}X_{k-2}\dots X_1X_0,$ $Y = Y_{k-1}Y_{k-2}\dots Y_1Y_0$

- Want to compute $Z = Z_{k-1}Z_{k-2}\dots Z_1Z_0$ where for each i , $Z_i = X_i + Y_i$

- Can just write $Z = X + Y$ as a shorthand (here $+$ is logical OR)

- e.g., $X = 0110, Y = 0011, Z = X + Y$

$$\begin{array}{r} 0110 \\ + 0011 \\ \hline \end{array} \quad \text{(bitwise logical OR)}$$

- e.g., $X = 0110, Y = 0011, Z = XY$

$$\begin{array}{r} 0110 \\ \cdot 0011 \\ \hline \end{array} \quad \text{(bitwise logical AND)}$$

Bit-wise (in parallel) Logic Ops

- Up to now, we looked at **single-bit** logic ops, e.g., $X + Y = Z$ where X, Y, Z are each one bit
- Will often need to perform logic op on **set of bits** in parallel, e.g.,

- $X = X_{k-1}X_{k-2}\dots X_1X_0,$ $Y = Y_{k-1}Y_{k-2}\dots Y_1Y_0$

- Want to compute $Z = Z_{k-1}Z_{k-2}\dots Z_1Z_0$ where for each i , $Z_i = X_i + Y_i$

- Can just write $Z = X + Y$ as a shorthand (here $+$ is logical OR)

- e.g., $X = 0110, Y = 0011, Z = X + Y$

$$\begin{array}{r} 0110 \\ + 0011 \\ \hline 0111 \end{array} \quad \text{(bitwise logical OR)}$$

- e.g., $X = 0110, Y = 0011, Z = XY$

$$\begin{array}{r} 0110 \\ \cdot 0011 \\ \hline \end{array} \quad \text{(bitwise logical AND)}$$

Bit-wise (in parallel) Logic Ops

- Up to now, we looked at **single-bit** logic ops, e.g., $X + Y = Z$ where X, Y, Z are each one bit
- Will often need to perform logic op on **set of bits** in parallel, e.g.,
 - $X = X_{k-1}X_{k-2}\dots X_1X_0$, $Y = Y_{k-1}Y_{k-2}\dots Y_1Y_0$
 - Want to compute $Z = Z_{k-1}Z_{k-2}\dots Z_1Z_0$ where for each i , $Z_i = X_i + Y_i$
 - Can just write $Z = X + Y$ as a shorthand (here $+$ is logical OR)
 - e.g., $X = 0110$, $Y = 0011$, $Z = X + Y$

$$\begin{array}{r} 0110 \\ + 0011 \\ \hline 0111 \end{array} \quad \text{(bitwise logical OR)}$$

$$\begin{array}{r} 0110 \\ \cdot 0011 \\ \hline 0011 \end{array} \quad \text{(bitwise logical AND)}$$

Bit-wise (in parallel) Logic Ops

- Up to now, we looked at **single-bit** logic ops, e.g., $X + Y = Z$ where X, Y, Z are each one bit
- Will often need to perform logic op on **set of bits** in parallel, e.g.,

- $X = X_{k-1}X_{k-2}\dots X_1X_0,$ $Y = Y_{k-1}Y_{k-2}\dots Y_1Y_0$

- Want to compute $Z = Z_{k-1}Z_{k-2}\dots Z_1Z_0$ where for each i , $Z_i = X_i + Y_i$

- Can just write $Z = X + Y$ as a shorthand (here $+$ is logical OR)

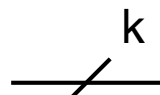
- e.g., $X = 0110, Y = 0011, Z = X + Y$

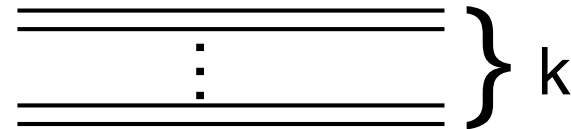
$$\begin{array}{r} 0110 \\ + 0011 \\ \hline 0111 \end{array} \quad \text{(bitwise logical OR)}$$

- e.g., $X = 0110, Y = 0011, Z = XY$

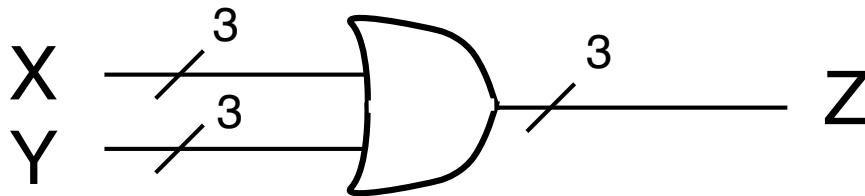
$$\begin{array}{r} 0110 \\ \cdot 0011 \\ \hline 0010 \end{array} \quad \text{(bitwise logical AND)}$$

Bitwise logical ops with circuitry

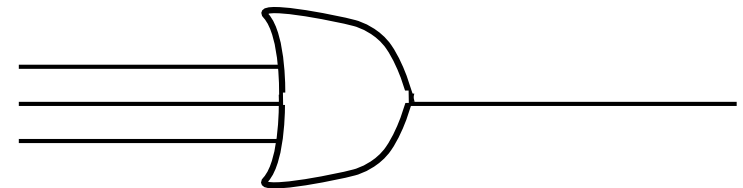
-  = k wires running in parallel, same as
- extends to circuits as well, e.g., k=3



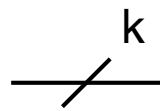
- $X = 101$, $Y = 110$, $X + Y = 111$

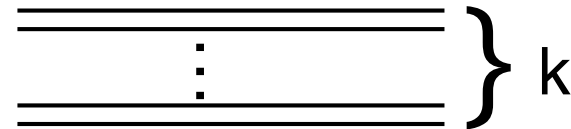


- Note the difference between above and which ORs 3 bits together

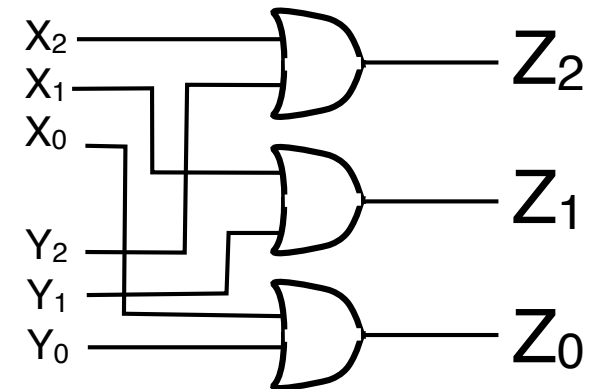
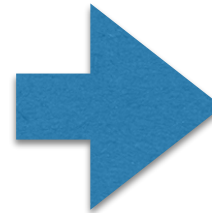
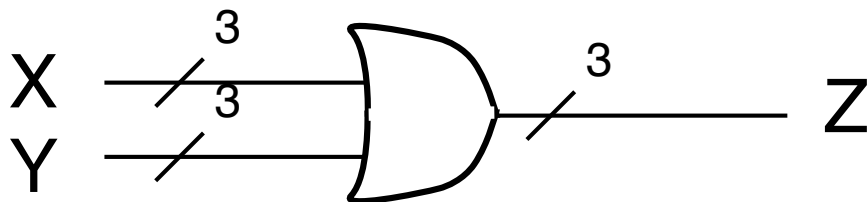


Bitwise logical ops with circuitry

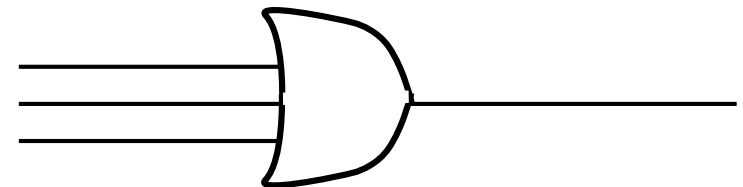
-  = k wires running in parallel, same as
- extends to circuits as well, e.g., k=3



- $X = 101$, $Y = 110$, $X + Y = 111$



- Note the difference between above and which ORs 3 bits together



NAND and NOR: Universal gates

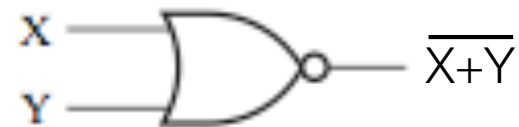
Universal gates: NAND, NOR

x	y	$z = \overline{xy}$
0	0	1
0	1	1
1	0	1
1	1	0



Note: the “o” in a circuit represents a NOT (inverter)

x	y	$z = \overline{x+y}$
0	0	1
0	1	0
1	0	0
1	1	0



Different from “●” which represents wire connector

NAND and NOR **universal** because...

- NOT, AND, OR can each be implemented using only NAND gates
- NOT, AND, OR can each be implemented using only NOR gates

$\bar{A} = A \text{ NAND } A$	$\bar{A} = A \text{ NOR } A$
$AB = \overline{A \text{ NAND } B}$	$A+B = \overline{A \text{ NOR } B}$
$A+B = \bar{A} \text{ NAND } \bar{B}$	$AB = \bar{A} \text{ NOR } \bar{B}$

Any circuit can be entirely implemented with just NAND or just NOR gates!!!

Duals

Duals

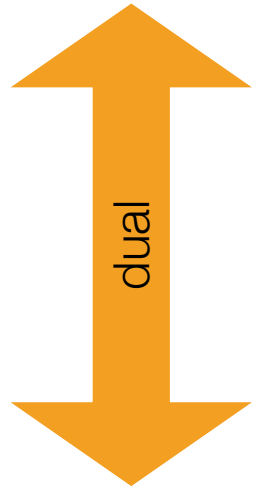
- All boolean expressions have **duals**
- Any theorem you can prove, you can also prove for its **dual**
- To form a dual...
 - replace AND with OR, OR with AND
 - replace 1 with 0, 0 with 1

What is the dual of this expression?

$$\overline{X} + \overline{Y} = \overline{XY}$$

What is the dual of this expression?

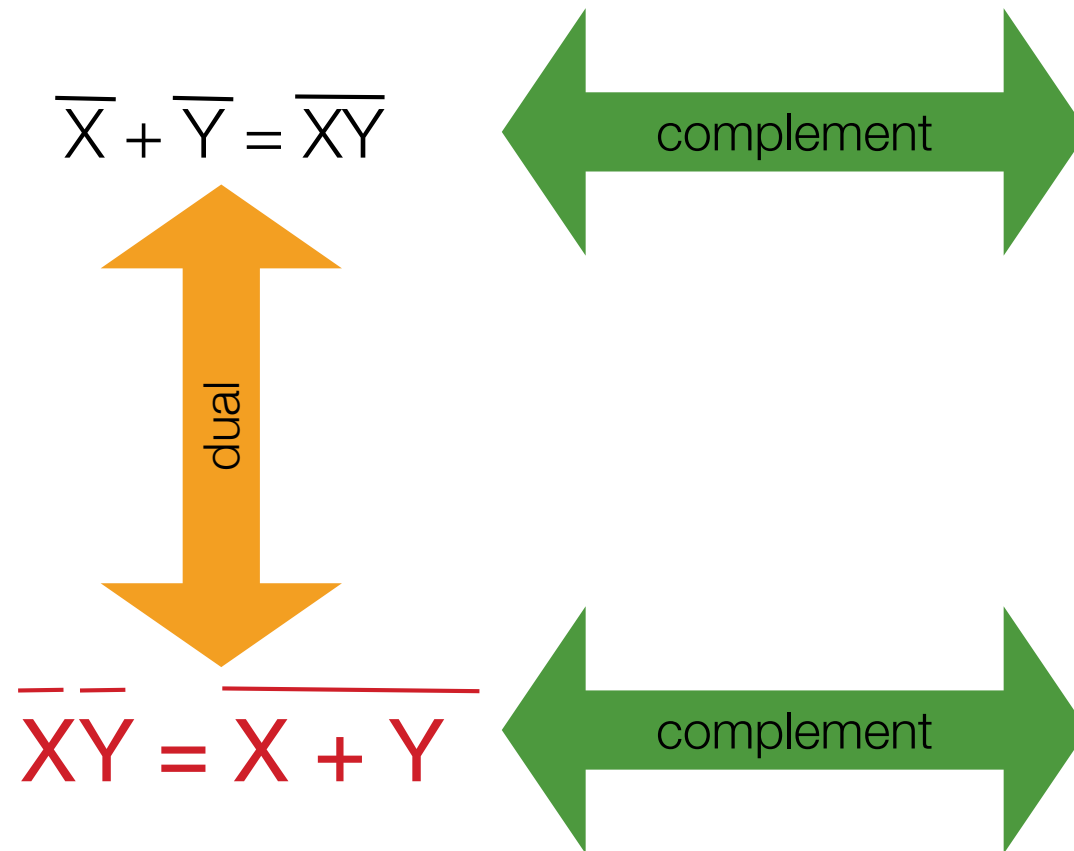
$$\overline{X} + \overline{Y} = \overline{XY}$$



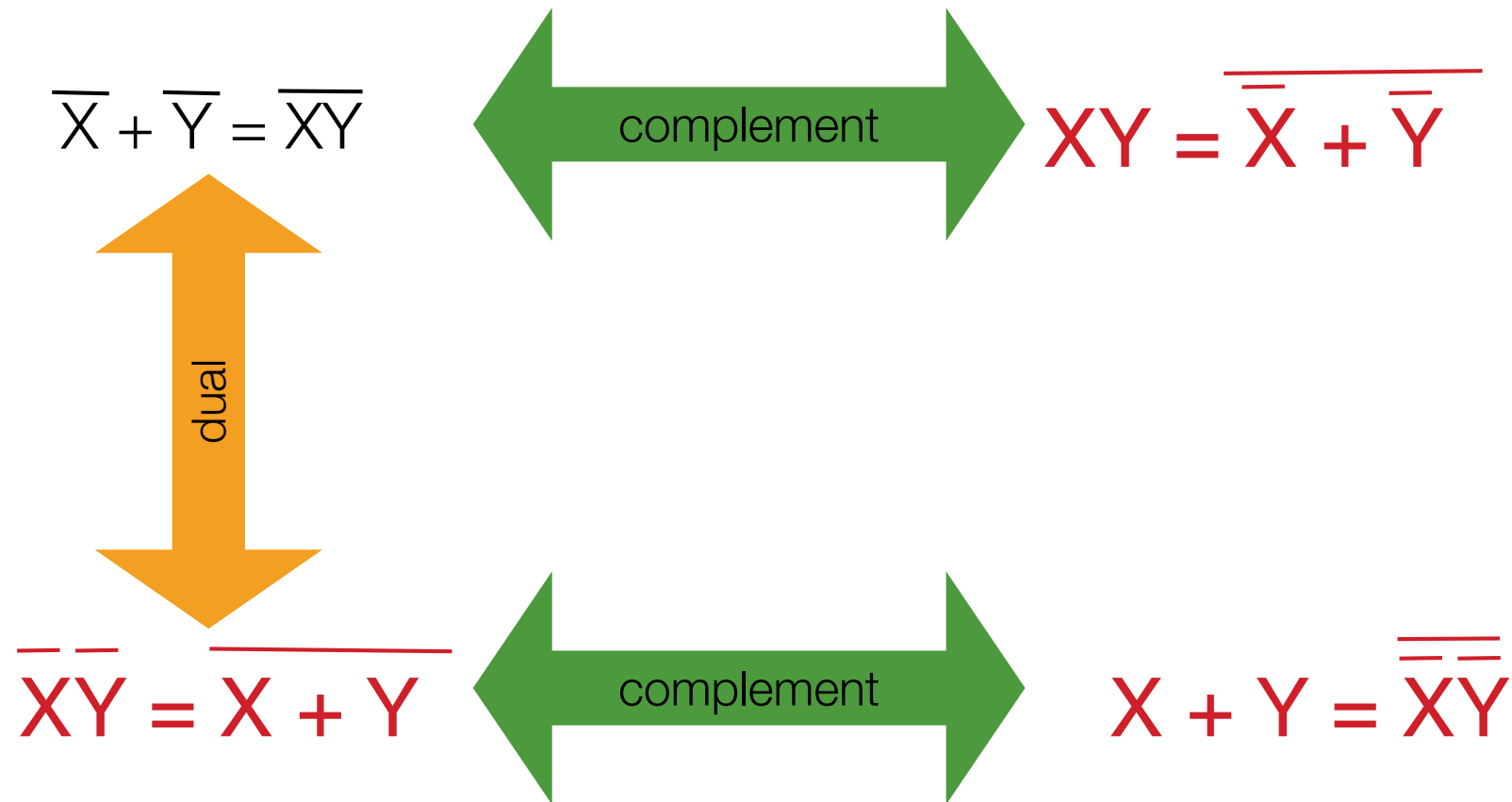
dual

$$\overline{\overline{X} \overline{Y}} = \overline{X + Y}$$

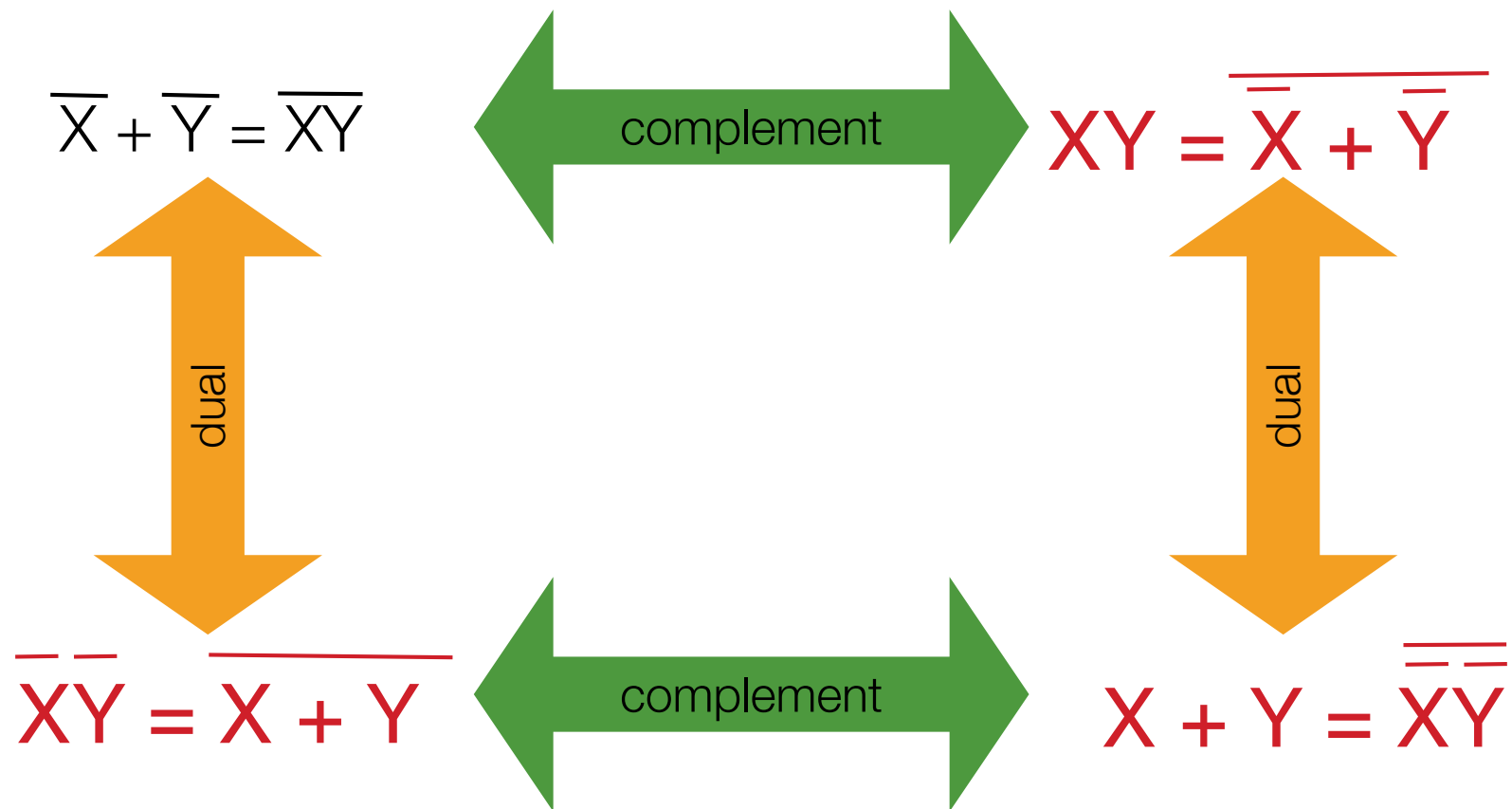
What are the complements of these expressions?



What are the complements of these expressions?



These are also the duals of one another.



Note: to complement a function, compute its dual,
then complement literals

“Complement using Dual” example

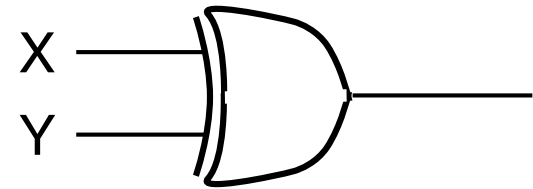
- $F = X + A(Z + \bar{X}(Y + W) + \bar{Y}(Z + W))$
- Dual: $F_{\text{dual}} = X(A + Z(\bar{X} + YW)(\bar{Y} + ZW))$ (swapped ANDs and ORs)
- $\bar{F} = \bar{X}(\bar{A} + \bar{Z}(X + \bar{Y}\bar{W})(Y + \bar{Z}\bar{W}))$ (“Flipped” literals)

Revisiting XOR

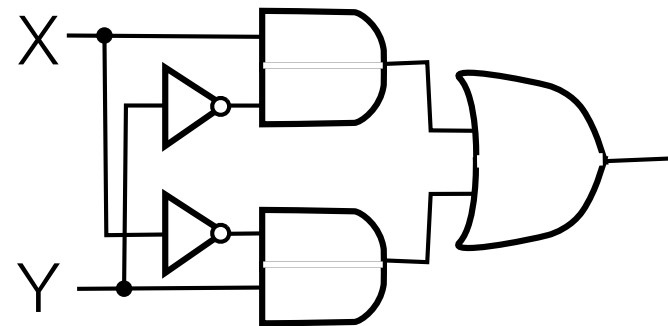
XOR: the parity operation

- $X \oplus Y = X\bar{Y} + \bar{X}Y$

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0



II



- In general, represents parity, i.e.,

- $X_1 \oplus X_2 \oplus X_3 \oplus \dots \oplus X_k = 1$ when an odd number of $X_i = 1$

Summary

- Subtle things that are very important to remember from this lecture:
 - Boolean order of precedence (slide 6)
 - XOR (slide 8)
 - Truth Tables (slide 10)
 - Boolean (basic) theorems (slide 15)
 - DeMorgan's Law (slide 21) & using to simplify complemented expressions (slide 50)